

AWS Batch ETL Pipeline - Complete Guide

AWS Batch ETL Pipeline

Complete Project Guide & Interview Handbook

Prepared for Sudama Pandit

SudamaPandit/aws-end-to-end-etl-pipeline

POSITIONING

The repository has passing unit CI and real implementation code, but it is not deployment-ready and has not been proven end-to-end against live PostgreSQL, Airflow, S3, Glue and Bedrock in CI.

Clickable Contents

1. Executive summary

2. Beginner foundations

3. Architecture diagram

4. End-to-end data flow

5. Repository structure

6. Folder-wise code walkthrough

7. Quality, reliability and security

Clickable Contents (continued)

8. Testing and CI evidence

9. Gaps and critical risks

10. Production-ready redesign

11. Fifty interview questions

12. Rapid revision sheet

13. Glossary

Executive Summary

What this project is

A scheduled batch ETL example moves bounded order data from PostgreSQL to an S3 raw prefix, transforms it with AWS Glue/PySpark into partitioned Parquet, and loads it into a PostgreSQL analytics table through an upsert. Airflow coordinates the runtime path, while pandas helpers make deterministic logic easy to unit-test.

Business value

Operational order tables should not carry heavy analytics workloads. This design separates ingestion, transformation and reporting, gives analysts a clean fact table, and creates intermediate storage that can support replay once run-specific immutable keys are added.

60-second interview introduction

I built a compact AWS batch ETL reference pipeline for orders. Airflow extracts a caller-defined date window from PostgreSQL, uploads raw CSV to S3, starts a Glue PySpark transformation, and loads curated Parquet into a PostgreSQL fact table with an order_id upsert. CI verifies the reusable transformation, quality rules, AI-disabled behavior and Glue polling states. I call it a portfolio implementation rather than production-ready because live integration, durable watermarking, quarantine, run isolation and infrastructure automation remain to be completed.

Honest status

The repository has passing unit CI and real implementation code, but it is not deployment-ready and has not been proven end-to-end against live PostgreSQL, Airflow, S3, Glue and Bedrock in CI.

Beginner Foundations

Use this section to learn the vocabulary before studying the code. A data pipeline moves data through controlled stages. ETL means extract from a source, transform into a useful and reliable shape, and load into a destination.

- > Batch processing handles bounded data on a schedule rather than continuously.
- > S3 prefixes act like logical folders; raw means minimally changed and curated means analytics-ready.
- > Parquet is typed, compressed and columnar, unlike the raw CSV landing file.
- > A watermark records the last successfully processed point; this repository accepts dates but does not persist the watermark.
- > Idempotency means a retry reaches the intended same state; the final upsert helps, but file-level concurrency is incomplete.

How to study the repository

First memorize the business flow, then trace one record through every component. Next learn which parts are actual runtime code, which are offline helpers, and which capabilities are missing. In interviews, evidence matters more than broad claims.

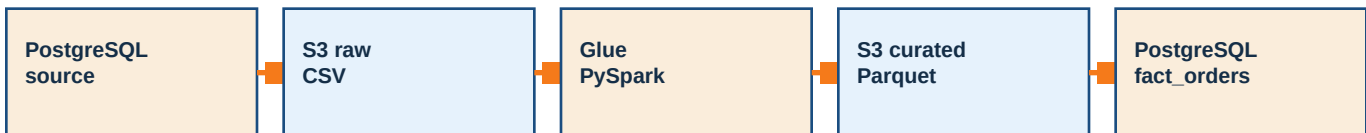
Architecture Diagram

The solid flow is implemented in code. Dashed or lighter notes mark external dependencies or incomplete production capabilities.

How to explain this diagram

- > PostgreSQL supplies a half-open date window of order records.
- > S3 raw decouples extraction from Glue, although the fixed orders.csv key is not truly immutable. Retries and bounded polling are implemented; live service integration is external.
- > Glue casts, cleans, deduplicates and partitions by order year/month.
- > The loader currently downloads curated Parquet into pandas and upserts PostgreSQL.
- > Bedrock is optional and advisory; aggregate metrics only are sent.

AIRFLOW CONTROL PLANE: extract -> transform -> load -> optional AI triage



End-to-End Data Flow

Step 1 - Schedule and configuration

Airflow schedules the DAG daily. Environment variables supply database URLs, S3 prefixes, Glue job name and extraction dates.

Step 2 - Extract

src/extract.py executes a parameterized SQL query with START_DATE inclusive and END_DATE exclusive, writes /tmp/orders.csv and returns its path.

Step 3 - Raw landing

src/aws_services.py verifies the file exists and uploads it to the configured S3 raw key.

Step 4 - Glue transformation

The DAG passes SOURCE_PATH and TARGET_PATH to Glue. Spark casts amount, parses dates, normalizes status, filters some invalid rows, deduplicates order_id and writes year/month-partitioned Parquet in overwrite mode.

Step 5 - Analytics load

src/load.py lists curated Parquet, downloads files, concatenates them in pandas, writes stg_fact_orders and performs PostgreSQL ON CONFLICT(order_id) DO UPDATE.

Step 6 - Advisory triage

The final optional task sends aggregate curated object count and byte metrics to Bedrock when configured.

Repository Structure

This table explains what every important folder contributes and whether it is used by the main runtime path.

Path	Purpose	Runtime truth
.github/workflows/ci.yml	Python setup, compile and pytest	Runs in CI; no live services
dags/etl_pipeline.py	Airflow task graph	Runtime entry; not DAG-import tested
glue/transform_job.py	Spark transformation	Intended Glue entry point
src/extract.py	PostgreSQL date-window extract	Called by DAG
src/aws_services.py	S3/Glue adapters and waiter	Called by DAG
src/transform.py	Pandas transformation	Offline helper; not DAG runtime
src/data_quality.py	Deterministic pandas gate	Tested; not wired to runtime
src/load.py	Parquet download and upsert	Called by DAG; not integration tested
src/ai_data_quality.py	Optional Bedrock summary	Called after load when configured
sql/	Tables, indexes and analytics	Manual/external application
config/	Example YAML	Not loaded by current runtime
tests/	Offline unit tests	Runs in CI

Folder-wise Code Walkthrough

dags/etl_pipeline.py

Defines four PythonOperator tasks and their dependency chain. Imports inside task functions delay heavy imports until execution.

```
extract_task >> glue_task >> load_task >> ai_task
schedule = "0 2 * * *"
retries = 2
```

Interview meaning: The graph is clear, but run-specific parameters and concurrency controls must be added.

src/extract.py

Uses a bound half-open time window so adjacent batches do not overlap at the boundary.

```
SELECT order_id, customer_id, order_date, amount, status
FROM source_orders
WHERE order_date >= %(start_date)s
AND order_date < %(end_date)s
```

Interview meaning: This is caller-controlled bounded extraction, not durable incremental state or CDC.

glue/transform_job.py

Spark performs distributed casting, normalization, filtering, deduplication and partition derivation.

```
withColumn("amount", col("amount").cast("decimal(18,2)"))
filter(col("order_id").isNotNull())
dropDuplicates(["order_id"])
partitionBy("order_year", "order_month")
```

Interview meaning: Glue rules are weaker than the pandas quality contract, which is the highest correctness gap.

src/data_quality.py

Fails closed on missing columns, empty data, invalid identifiers, amounts, dates and statuses.

```
failed = [name for name, passed in checks.items() if not bool(passed)]
if failed:
    raise ValueError(f"Data-quality checks failed: {failed}")
```

Interview meaning: The logic is tested but not wired before curated publication.

src/load.py

Stages selected columns and upserts by order_id.

Continued

```
INSERT INTO fact_orders (...)  
SELECT ... FROM stg_fact_orders  
ON CONFLICT (order_id) DO UPDATE SET  
amount = EXCLUDED.amount,  
loaded_at = CURRENT_TIMESTAMP
```

Interview meaning: The final table avoids duplicate IDs, but listing, pandas memory and a shared staging table do not scale safely.

src/aws_services.py

The Glue waiter recognizes terminal failures and has a bounded timeout.

```
if state == "SUCCEEDED": return  
if state in {"FAILED", "STOPPED", "TIMEOUT", "ERROR"}: raise RuntimeError  
raise TimeoutError(...)
```

Interview meaning: A deferrable Glue sensor would be more worker-efficient in production.

Quality, Reliability and Security

Idempotency

PostgreSQL ON CONFLICT makes repeated order IDs update rather than duplicate. Fixed S3 keys, overwrite mode and a shared staging table mean the entire run is not concurrency-safe.

Data quality

The pandas gate is strong for the compact schema, but it is not executed by the DAG. Glue also omits several equivalent rules.

Partitioning

Year/month partitions support time pruning, but the correct grain depends on data volume and query patterns.

Failure handling

Airflow retries tasks and the Glue waiter handles terminal states and timeout. There is no quarantine or run manifest.

Scalability

Downloading all Parquet into pandas is the first major bottleneck. Production should use distributed or bulk loading.

Configuration

Environment variables drive runtime. The YAML file is currently documentation rather than an active configuration source.

Security baseline

No real credentials should be stored in Git. Runtime identities should use least-privilege roles, secrets should come from a managed secret backend, logs should avoid raw personal data, and CI permissions should remain read-only unless a workflow genuinely needs write access.

Testing and CI Evidence

A passing CI badge proves only the checks that CI actually executes. It is not proof of deployment or end-to-end service integration.

Area	Evidence	What it does not prove
Pandas transform	Fixture assertions for cleaning, dates, amount and deduplication	Glue equivalence
Quality gate	Valid and invalid DataFrames raise named results/errors	Runtime enforcement
AI safety	Disabled path and prompt content	Bedrock response/auth behavior
Glue waiter	Fake client success and terminal failure	Real Glue execution
CI compile	src Python compiles on Python 3.12	Airflow DAG imports
End to end	Not present	PostgreSQL-S3-Glue-target correctness

Best interview wording

I tested the deterministic Python components offline: pandas transformation and quality cases, the safe disabled AI path, prompt construction, and Glue waiter state handling with a fake client. CI passes those checks. I did not treat that as proof of live PostgreSQL, Airflow, S3 or Glue execution; I would add Testcontainers, local Spark contract tests, DAG-bag validation and a staging AWS smoke test.

Tests to add next

- > PostgreSQL Testcontainers test for extraction and repeated upsert.
- > Local Spark test using the exact Glue transformations and invalid fixtures.
- > Airflow DagBag import and dependency test.
- > botocore Stubber tests for S3 and Glue request contracts.
- > Small AWS staging run with source-to-target reconciliation.

Gaps and Critical Risks

These are not reasons to hide the project. They are the strongest discussion points because they show that you can distinguish a portfolio implementation from a production system.

Severity	Risk	Consequence / response
Critical	Pandas quality gate is not in runtime path	Bad customer IDs, dates or statuses can reach curated output; enforce one canonical Spark contract.
High	Fixed raw key and overwrite target	Concurrent runs can overwrite each other; use run-specific paths and atomic promotion.
High	Unbounded pandas load	Memory/local disk failure at scale; use Spark, COPY or warehouse-native load.
High	S3 listing is not paginated	Files beyond the first page can be missed; use paginator or manifest.
High	No durable watermark or CDC	Gaps, overlaps and deletes are caller-managed; add a control table or CDC.
Medium	Shared staging table	Concurrent loaders interfere; use unique staging and transactional publication.
Medium	No quarantine	Rejected records are not recoverable; store reason-coded rejects.
Medium	Heavy loose dependencies	Builds can drift or slow; split and pin environments.

Production-Ready Redesign

Phase 1: Correctness first

Define an explicit schema, unify pandas/Spark rules, wire the gate into Glue, write a quarantine dataset and reconcile counts.

Phase 2: Run isolation

Use logical date/run ID paths, unique staging tables, manifests, controlled overwrite and Airflow concurrency limits.

Phase 3: Integration evidence

Add PostgreSQL containers, local Spark, DAG import checks, client stubs and a small real staging test.

Phase 4: Infrastructure and operations

Provision S3, IAM, Glue, logging, alarms and secret access through Terraform; add run manifests, SLAs and runbooks.

Phase 5: Scale and CDC

Replace pandas loading, compact files, paginate listings and adopt DMS/Debezium or durable watermark state.

How to answer design follow-ups

Use Context -> Decision -> Trade-off -> Evidence -> Next step. State what the repository demonstrates, cite the exact test or implementation, acknowledge the boundary, and explain the production change. Never invent deployment history.

Fifty Interview Questions

The answers below are written as spoken interview responses. Learn the ideas rather than memorizing every word.

1 Explain the project end to end.

Interview-ready answer: Airflow extracts a bounded order window from PostgreSQL, uploads CSV to S3 raw, starts Glue to create partitioned Parquet, then loads the curated files into fact_orders with an upsert. An optional Bedrock step summarizes aggregate operational metrics.

2 Why use a half-open date range?

Interview-ready answer: Using start inclusive and end exclusive lets adjacent windows meet without double-counting the boundary timestamp. The current dates are caller-controlled; I would persist the last successful watermark in production.

3 Where is watermark state?

Interview-ready answer: It is not implemented. I call the current approach bounded extraction rather than true stateful incremental loading, and I would add a control table advanced only after successful reconciliation.

4 Is raw S3 really immutable?

Interview-ready answer: Not yet, because the DAG writes a fixed orders.csv key. I would use source/date/run-specific keys and versioning or write-once controls.

5 How are duplicates handled?

Interview-ready answer: Glue drops duplicate order_id values inside the input and PostgreSQL upserts on order_id. I still need a deterministic rule for which duplicate wins, preferably based on source update sequence.

6 How are deletes handled?

Interview-ready answer: They are not propagated. I would use CDC tombstones or periodic source-target reconciliation with a soft-delete strategy.

7 Why is the quality gate a concern?

Interview-ready answer: The pandas gate is tested but the DAG does not call it, and Glue applies only a subset. My first correctness change would enforce the canonical rules in Glue before publication.

8 What happens if Glue hangs?

Interview-ready answer: The waiter polls with a timeout and raises TimeoutError. In production I would use an Airflow deferrable sensor to avoid occupying a worker.

Interview Questions (continued)

9 Can two DAG runs overlap safely?

Interview-ready answer: No. They share local paths, S3 keys, overwrite targets and the staging table. I would isolate every run and add concurrency controls.

10 Why use Parquet?

Interview-ready answer: It preserves types, compresses well and supports column and partition pruning, which reduces analytical scan cost compared with CSV.

11 What happens after 1,000 S3 objects?

Interview-ready answer: The current `list_objects_v2` call does not paginate, so later objects can be ignored. I would use a paginator or load from an explicit manifest.

12 Why is pandas loading risky?

Interview-ready answer: The loader downloads every curated file and concatenates it in one process, so local disk and memory become bottlenecks. I would use Spark/JDBC or bulk loading for meaningful scale.

13 How would you prove idempotency?

Interview-ready answer: I would load the same batch twice into ephemeral PostgreSQL and assert stable row count and values, then change one source record and verify only that row updates.

14 Why is Bedrock included?

Interview-ready answer: Only as an optional advisory layer for aggregate metric summaries. Deterministic checks remain authoritative and ETL should succeed even if AI is disabled.

15 Would you put AI in the critical path?

Interview-ready answer: No. A model should not decide whether data is correct or published. It can reduce investigation time, but rule-based gates control outcomes.

16 What does CI actually run?

Interview-ready answer: Python 3.12 setup, dependency installation, src compilation and pytest. It does not provision or call live infrastructure.

17 Where is Terraform?

Interview-ready answer: It is not included. The AWS resources must exist externally, so infrastructure-as-code is a required production improvement.

18 How would you backfill one month?

Interview-ready answer: I would split it into bounded daily windows, give each run isolated paths, control parallelism, upsert idempotently and reconcile every completed slice.

Interview Questions (continued)

19 What is the first production fix?

Interview-ready answer: Wire a canonical Spark quality gate with explicit schema and quarantine, because correctness must be established before scaling or adding features.

20 How would you explain the project without overclaiming?

Interview-ready answer: I say it demonstrates the batch design and tested Python behavior, while live integration and deployment are not yet proven. Then I explain the exact tests needed to close that gap.

21 What is the benefit of staging before upsert?

Interview-ready answer: It separates bulk ingestion from merge logic and allows validation before touching the target. The current shared staging table needs run-specific isolation.

22 How would you optimize monthly queries?

Interview-ready answer: Keep date-oriented partitioning upstream, index appropriate target columns, inspect EXPLAIN ANALYZE and consider summary tables if repeated aggregation becomes expensive.

23 What happens on an invalid amount?

Interview-ready answer: The pandas gate rejects non-numeric or negative values. Glue casts and removes negative values, but nulls created by failed casts need an explicit rejection/quarantine rule.

24 Why import inside Airflow task functions?

Interview-ready answer: It reduces DAG parse-time dependency work and keeps execution imports close to tasks. I would still verify the DAG with DagBag in CI.

25 What would fail first at one terabyte?

Interview-ready answer: The pandas-based loader and local downloads would fail first. Transformation can scale with Spark, but the load path must also be distributed or bulk-oriented.

26 Did you deploy this project?

Interview-ready answer: I describe it as a working portfolio implementation with passing unit CI, not as a production deployment. The repository proves specific code paths; a staging environment and integration evidence are still required.

27 What exactly did you test?

Interview-ready answer: I tested pandas transformations and quality failures, AI disabled behavior and prompt content, and Glue waiter success/failure states. Those tests run offline and isolate deterministic behavior, so I do not claim they validate live external services.

Interview Questions (continued)

28 Why does green CI not prove the platform works?

Interview-ready answer: CI executes syntax and unit tests, but it does not automatically prove networking, IAM, service configuration, runtime compatibility or end-to-end data correctness. I would add integration and staging smoke tests for that evidence.

29 What is the biggest gap?

Interview-ready answer: The most important gap is the mismatch between the tested pandas quality gate and the actual Glue runtime path. I would address it before calling the system production-ready.

30 How do you handle secrets?

Interview-ready answer: The repository should contain examples only. I would use workload identities or IAM roles for cloud access and a managed secret backend for database credentials, with least privilege and rotation.

31 How do you handle logging?

Interview-ready answer: I would use structured logs with run or batch identifiers, input and output counts, durations, quality results and error categories. I would avoid raw customer data and make logs searchable in a central platform.

32 How do you monitor data freshness?

Interview-ready answer: I would record the newest processed event or business timestamp and compare it with the freshness SLA. Breaches should alert with the affected dataset and last successful run.

33 How do you make retries safe?

Interview-ready answer: Every side effect needs an idempotency key or deterministic batch identifier. I would isolate run outputs, use upserts or transactional publication, and ensure a retry cannot duplicate or overwrite another run.

34 How do you support backfills?

Interview-ready answer: I would parameterize a bounded time range, use run-specific outputs, limit concurrency, reconcile every partition, and record completion in a control table so a failed slice can be resumed safely.

35 How do you prevent schema drift?

Interview-ready answer: I would define an explicit versioned contract, validate it at ingestion, quarantine incompatible records, test producer-consumer compatibility and allow only reviewed evolution.

36 How would you improve CI?

Interview-ready answer: I would keep fast unit checks, add linting and dependency scanning, then add targeted integration jobs using ephemeral infrastructure. Live staging smoke tests should run separately with tightly scoped credentials.

Interview Questions (continued)

37 How would you test external services?

Interview-ready answer: I would combine client stubs for request contracts, local or containerized substitutes where accurate, and a small real staging test for service-specific behavior that emulators cannot prove.

38 How do you protect customer data?

Interview-ready answer: I would minimize collected fields, encrypt in transit and at rest, restrict access by role and prefix/table, mask sensitive fields, set retention policies and audit access.

39 How do you estimate capacity?

Interview-ready answer: I would measure arrival rate, record size, daily volume, peak concurrency, transformation cost and downstream throughput. Those numbers drive partitioning, compute sizing and retention choices.

40 What happens when data quality fails?

Interview-ready answer: Production should stop publication for critical failures, persist rejected records with reason codes, alert the owner and support corrected replay. Warning-level rules can continue with explicit metrics.

41 How do you prove lineage?

Interview-ready answer: I would attach run ID, source window, input location, code version and output location to a run manifest and integrate those records with a catalog or lineage platform.

42 What would you document for operations?

Interview-ready answer: I would document SLAs, owners, dependencies, restart and backfill procedures, common failure signatures, data contracts, access requirements and rollback steps.

43 What would fail first at scale?

Interview-ready answer: The first bottleneck is likely the all-files-to-local-pandas loader. I would validate that with load tests and redesign before increasing volume.

44 How do you control cost?

Interview-ready answer: I would right-size compute, prune partitions, compact small files, apply storage lifecycle rules, cap retries and timeouts, and publish per-run cost and volume metrics.

45 What would you change in your first week?

Interview-ready answer: I would first add evidence around the highest-risk path: wire canonical Spark validation/quarantine, isolate run paths, paginate S3 and add PostgreSQL/local-Spark integration tests. Then I would update the runbook and architecture based on measured results.

Interview Questions (continued)

46 How do you explain a limitation without weakening your interview?

Interview-ready answer: I state the boundary directly, explain why the current choice was reasonable for a portfolio scope, cite what is actually tested, and give a concrete production solution.

47 Why should an interviewer value this project?

Interview-ready answer: It shows that I understand the complete data lifecycle and can review my own design critically. I can discuss implementation, tests, failure modes and realistic hardening rather than only drawing an architecture.

48 How would you perform a production readiness review?

Interview-ready answer: I would review correctness, recoverability, security, scalability, observability, deployment, cost and ownership. Each requirement needs evidence, not only code presence.

49 What is your rollback strategy?

Interview-ready answer: I would version code and data outputs, publish atomically, retain the previous valid version, and make consumers switch through a controlled pointer or view. Rollback steps must be tested.

50 Give the honest one-line status.

Interview-ready answer: This is a compact, technically useful portfolio implementation with verified unit behavior and clearly documented production gaps; it is not a fully deployed platform.

Rapid Revision Sheet

One-minute memory map

- > Flow: PostgreSQL -> S3 raw CSV -> Glue -> S3 curated Parquet -> PostgreSQL fact table.
- > Orchestrator: Airflow with retries and bounded Glue polling.
- > Incremental truth: caller-defined dates, not persisted watermark or CDC.
- > Idempotency truth: final upsert helps; file and concurrency safety remain incomplete.
- > Testing truth: offline unit CI, no live end-to-end proof.
- > Top risk: canonical quality rules are not wired into the runtime path.

Safe sentence when challenged

The repository demonstrates the design and unit-tests specific offline behavior. It does not prove every external integration or a production deployment. I can show exactly what is verified and what I would add to prove the remaining path.

Glossary

Term	Meaning
Airflow	Workflow scheduler and dependency orchestrator.
Glue	AWS managed Spark-based data integration service.
Parquet	Compressed columnar storage format.
Watermark	Stored progress point for incremental processing.
CDC	Change data capture for inserts, updates and deletes.
Upsert	Insert a new record or update an existing match.
Quarantine	Durable rejected records plus reason codes.
Idempotency	A retry reaches the intended same state.
Reconciliation	Compare source, intermediate and target metrics.
Partition pruning	Read only partitions relevant to a query.